

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 4

Roll no: 08 PRN No: 12412097 Name of Student: Sahil Balasaheb Patil

Subject Name : Deep Learning

Problem Statement :

Develop and LSTM-based model for time-series forecasting using stock-price weather or sales datasets.

Algorithm :

1. Start
2. Load the Stock Price Dataset (e.g., historical OHLC data — Open, High, Low, Close, Volume) for a given company/ticker.
3. Identify the target variable (e.g., Close Price) to forecast.
4. Preprocess the data:
 - Handle missing values (forward-fill / drop)
 - Sort data chronologically by date
 - Select relevant feature(s) (e.g., 'Close' column)
 - Normalize values using MinMaxScaler (scale 0–1) since LSTMs are sensitive to input scale
5. Create sequences (sliding window) from the time series:
 - Use past N time steps (e.g., 60 days) to predict the next time step
 - Convert 1D series into supervised learning format: $X = [t-60...t-1]$, $y = [t]$
6. Split the dataset into:
 - 80% Training data
 - 20% Testing data (maintaining temporal order — no shuffling!)

7. Reshape input data into 3D format required by LSTM: [samples, timesteps, features]
8. Initialize the LSTM architecture:
 - Input layer (shape = timesteps $\times$ features)
 - LSTM layer with 50 units (return_sequences=True)
 - Dropout layer with 0.2
 - LSTM layer with 50 units (return_sequences=False)
 - Dropout layer with 0.2
 - Fully connected Dense layer with 25 neurons
 - Output Dense layer with 1 neuron (regression — predicted price)
9. Compile the LSTM using:
 - Optimizer: Adam
 - Loss function: Mean Squared Error (MSE)
 - Evaluation metric: Mean Absolute Error (MAE)
10. Train the LSTM using the training dataset for a specified number of epochs with batching.
11. Validate the model after each epoch using the validation split.
12. Evaluate model performance using test loss (MSE) and MAE.
13. Generate predictions for the test dataset.
14. Inverse-transform predictions and actual values back to the original price scale.
15. Calculate error metrics: RMSE, MAE, and MAPE (Mean Absolute Percentage Error).
16. Plot Actual vs Predicted prices to visualize forecasting accuracy.
17. Save the trained LSTM model for future forecasting.
18. Input the most recent N-day window of new data and preprocess it the same way (scale, reshape).
19. Pass the window through the trained LSTM.
20. Predict the next time step's price.
21. Display the predicted price and de-normalized confidence range.
22. Stop

Code :

```
# =====
# LSTM-based Time-Series Forecasting (Stock Price Prediction)
# =====

# !pip install yfinance tensorflow scikit-learn matplotlib -q

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam

SEED = 42
np.random.seed(SEED)

WINDOW_SIZE = 60 # past 60 days used to predict next day
TICKER = "AAPL"

# -----
# 1. Load Dataset (Stock Price data via yfinance)
# -----
df = yf.download(TICKER, start="2015-01-01", end="2024-12-31")
df = df[["Close"]].dropna()

print("Total records:", len(df))
print(df.head())

# -----
# 2. Preprocess — Normalize
# -----
scaler = MinMaxScaler(feature_range=(0, 1))
```

```

scaled_data = scaler.fit_transform(df[["Close"]])

# -----
# 3. Train/Test Split (temporal — no shuffling)
# -----
TRAIN_RATIO = 0.8
train_size = int(len(scaled_data) * TRAIN_RATIO)

train_data = scaled_data[:train_size]
test_data = scaled_data[train_size - WINDOW_SIZE:] # include lookback window

# -----
# 4. Create sliding-window sequences
# -----
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(window_size, len(data)):
        X.append(data[i - window_size:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_data, WINDOW_SIZE)
X_test, y_test = create_sequences(test_data, WINDOW_SIZE)

# Reshape into [samples, timesteps, features]
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)

# -----
# 5. Build LSTM architecture
# -----
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)),
    Dropout(0.2),

```

```

    LSTM(50, return_sequences=False),
    Dropout(0.2),
    Dense(25, activation="relu"),
    Dense(1) # regression output — predicted price
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mean_squared_error",
    metrics=["mae"]
)

model.summary()

# -----
# 6. Train the model
# -----
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=25,
    batch_size=32,
    verbose=1
)

# -----
# 7. Evaluate on Test Set
# -----
test_loss, test_mae = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Loss (MSE): {test_loss:.6f}")
print(f"Test MAE      : {test_mae:.6f}")

# -----
# 8. Predict + Inverse Transform to original scale
# -----
predictions = model.predict(X_test)
predictions_actual = scaler.inverse_transform(predictions)

```

```

y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

# -----
# 9. Error Metrics (RMSE, MAE, MAPE)
# -----
rmse = np.sqrt(mean_squared_error(y_test_actual, predictions_actual))
mae = mean_absolute_error(y_test_actual, predictions_actual)
mape = np.mean(np.abs((y_test_actual - predictions_actual) / y_test_actual)) * 100

print(f"\nRMSE: {rmse:.4f}")
print(f"MAE : {mae:.4f}")
print(f"MAPE: {mape:.2f}%")

# -----
# 10. Plot Actual vs Predicted
# -----
plt.figure(figsize=(12, 6))
plt.plot(y_test_actual, label="Actual Price", color="blue")
plt.plot(predictions_actual, label="Predicted Price", color="red")
plt.title(f'{TICKER} Stock Price Prediction (LSTM)')
plt.xlabel("Time Step")
plt.ylabel("Price ($)")
plt.legend()
plt.show()

# -----
# 11. Save the trained model
# -----
model.save("lstm_stock_forecast_model.h5")

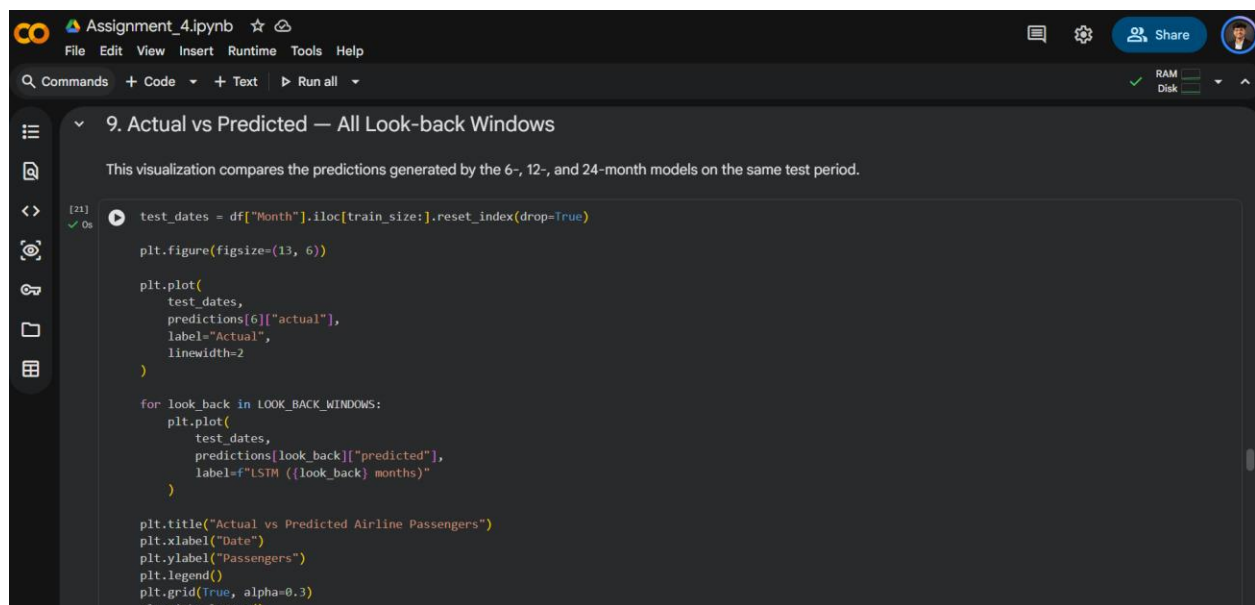
# -----
# 12. Inference — Predict the NEXT day's price
# -----
def predict_next_price(model, recent_data, scaler, window_size=WINDOW_SIZE):
    last_window = recent_data[-window_size:]
    scaled_window = scaler.transform(last_window.reshape(-1, 1))
    X_input = scaled_window.reshape(1, window_size, 1)

```

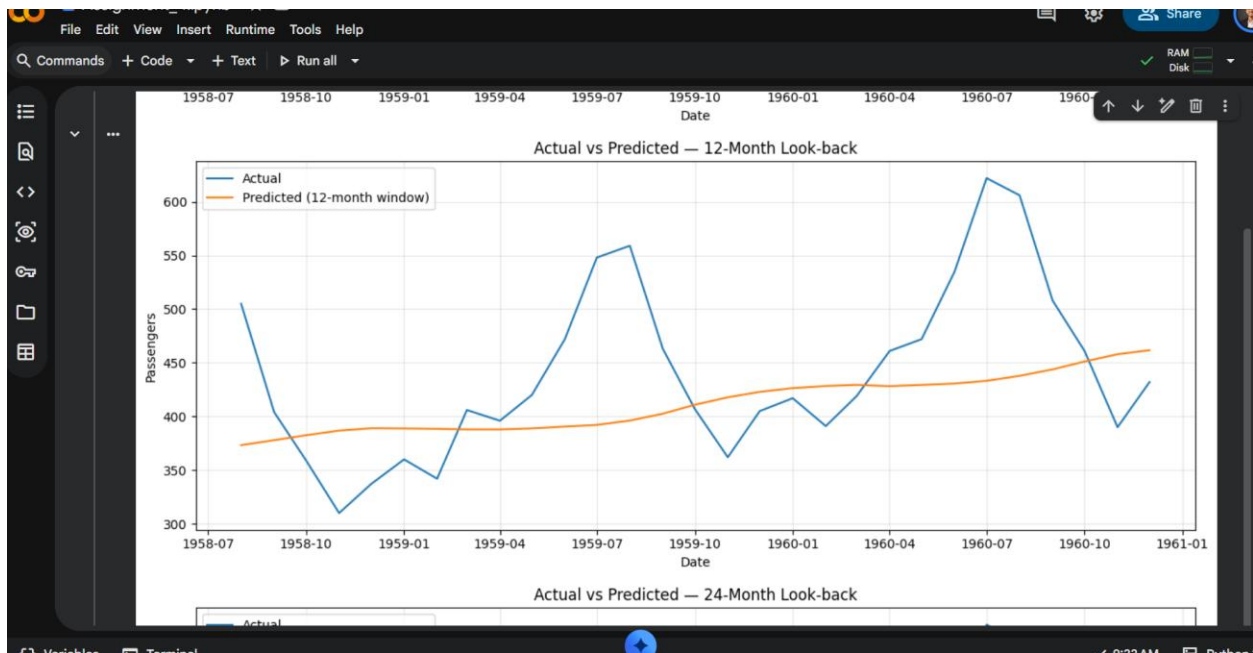
```
predicted_scaled = model.predict(X_input, verbose=0)
predicted_price = scaler.inverse_transform(predicted_scaled)
return predicted_price[0][0]
```

```
# Example: use the last 60 known closing prices
recent_prices = df["Close"].values[-WINDOW_SIZE:]
next_day_price = predict_next_price(model, recent_prices, scaler)
print(f"\nPredicted next day's closing price for {TICKER}: ${next_day_price:.2f}")
```

Output :



```
Assignment_4.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
9. Actual vs Predicted — All Look-back Windows
This visualization compares the predictions generated by the 6-, 12-, and 24-month models on the same test period.
[21] test_dates = df["Month"].iloc[train_size:].reset_index(drop=True)
      plt.figure(figsize=(13, 6))
      plt.plot(
          test_dates,
          predictions[6]["actual"],
          label="Actual",
          linewidth=2
      )
      for look_back in LOOK_BACK_WINDOWS:
          plt.plot(
              test_dates,
              predictions[look_back]["predicted"],
              label=f"LSTM ({look_back} months)"
          )
      plt.title("Actual vs Predicted Airline Passengers")
      plt.xlabel("Date")
      plt.ylabel("Passengers")
      plt.legend()
      plt.grid(True, alpha=0.3)
      plt.tight_layout()
```



Assignment_4.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

6-Month Window

Uses relatively recent history. It may capture short-term changes but does not contain a complete yearly seasonal cycle.

12-Month Window

Contains one complete year of monthly observations. This makes it a natural choice for a dataset with yearly seasonality.

24-Month Window

Provides two complete yearly cycles and therefore gives the model more historical context. However, the longer sequence also increases the amount of information the model must process.

The final choice should be based on the **observed MAE and RMSE**, rather than assuming that a particular window will always perform best.

13. Conclusion

Three LSTM forecasting models were trained using 6-, 12-, and 24-month look-back windows. Their performance was compared using MAE and RMSE.

The experiment demonstrates how the amount of historical context affects time-series forecasting performance. Since the Airline Passengers dataset contains strong yearly seasonality, the 12-month window is particularly meaningful, while the 6- and 24-month windows provide useful shorter- and longer-context comparisons.

The best configuration should be selected based on the experimental evaluation results.

